



UNIVERSIDAD DE SEVILLA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Grado en Ingeniería Informática – Tecnologías Informáticas

Curso 2025/2026

PROCESAMIENTO DE IMÁGENES AÉREAS PARA LA DETECCIÓN Y DIFERENCIACIÓN DE VEHÍCULOS

DOCUMENTACIÓN PARA LA ASIGNATURA DE PID

Realizado por:

Julio Nieto Pérez
Miguel Muñoz Barrios
Pablo Rendón Causín

Profesora:

María José Jiménez Rodríguez

Resumen

El planteamiento tras este trabajo es el entrenamiento del modelo neuronal YOLOv8 (“ultralytics/ultralytics: Ultralytics YOLO 🚀”, [1]) modificado para la detección a través de imágenes satélite de distintos vehículos que circulan por diferentes vías, basándonos en los siguientes artículos científicos: Vehicle Detection in Aerial Images Using Modified YOLO (Bin Xu et al. [2]), CoT-YOLOv8: Improved YOLOv8 for Aerial images Small Target Detection (Wang et al., n.d., [3]) y Robust Vehicle Detection in Aerial Imagery: Comparative Study of YOLO Variants (Kulkarni et al., n.d., [4]). Trabajaremos con Python, un lenguaje recomendado para la implementación del modelo que vamos a usar, a través de Google Colab aplicando nuestros conocimientos de procesamiento de imágenes e inteligencia artificial para poder entrenar el modelo YOLOv8.

Utilizaremos para esto el dataset VEHICLE DETECTION Dataset (sardar vallabhbhai national institute of technology, [5])

1. INTRODUCCIÓN

En los últimos años hemos podido ser testigos de la proliferación de la inteligencia artificial y como va avanzando y desarrollándose a un ritmo espectacular, habiendo mejoras en todas de las ramas ya existentes de esta y de otras nuevas.

Una de estas ramas es la detección de objetos, que detecta y clasifica distintos objetos de interés en una imagen, siendo capaz de localizar dichos objetos con unas coordenadas específicas (bounding box) y darle cierta clasificación. Esta rama se ha estado utilizando para diferentes aplicaciones en el mundo real, desde la seguridad y vigilancia hasta su uso como herramienta para estudios estadísticos.

Las arquitecturas de aprendizaje profundo desarrolladas en los últimos años, así como también las redes convolucionales han permitido desarrollar cada vez modelos de detección de objetos cada vez más precisos. Entre estos tenemos las redes *You Only Look Once* (YOLO) que se caracterizan por su rapidez y capacidad de procesamiento en tiempo real, por lo que son excelentes para el análisis instantáneo de grandes volúmenes de datos en imagen.

La detección de objetos en imágenes satelitales puede llegar a convertirse en un problema no tan fácil de resolver debido a los tamaños, resoluciones y la perspectiva de los objetos de la imagen. Los objetos suelen ser chicos en las imágenes y con diferentes contextos e incluso iluminación. Además, las imágenes aéreas y satelitales con un nivel alto de detalle necesitan de un modelo que pueda generalizar bien a partir de datos complejos y heterogéneos.

Este trabajo se centra en el entrenamiento e implementación de un modelo de detección de vehículos en imágenes satelitales a partir de la red no entrenada de YOLOv8. El objetivo es elaborar un modelo que sea capaz de identificar y distinguir diferentes tipos de vehículos en imágenes satelitales que sea útil como herramienta para diferentes aplicaciones.

2. PLANTEAMIENTO TEÓRICO

La detección de objetos como ya hemos explicado anteriormente es una rama del aprendizaje profundo. Esta combina la clasificación y localización de objetos dentro de una imagen. A diferencia de el reconocimiento de imagen que solo sabe identificar si hay presencia de una clase, esta puede determinar dónde se encuentra cada objeto y a qué categoría pertenece.

2.1. Fundamentos de la detección de objetos

1. **Clasificación de objetos:** Asignarle una etiqueta a cada objeto que se detecta
2. **Localización de objetos:** Determinar las coordenadas de la imagen que delimitan al objeto

Las coordenadas de la localización se suelen marcar con *Bounding Boxes*, que son rectángulos con coordenadas (x, y, ancho, alto) que la representan. Son prácticamente esenciales para:

- Entrenar modelos de deep learning
- Evaluar las métricas del desempeño de estos (precisión, recall, mAP)
- Aplicaciones reales en las que también se necesitan saber las coordenadas de estos objetos

2.2. Representación matemática

Las *Bounding Boxes* pueden representarse como:

- **(xmin,ymin,xmax,ymax)** indican los extremos en coordenadas absolutas del rectángulo
- **(xc,yc,w,h)** donde los dos primeros parámetros indican el punto centro del rectángulo y los dos últimos el ancho y alto respectivamente. Estas son las coordenadas normalizadas.

Estos modelos predicen las coordenadas junto con la probabilidad de cada clase. Tenemos una función de pérdida que tiene en cuenta los errores de **predicción de clase (clasificación)** y los **errores de localización (coordenadas bounding boxes)** para optimizar a la misma vez la exactitud y la posición. También se tiene en cuenta la probabilidad de que el objeto esté en esa posición.

2.3. Arquitecturas de detección

En los modelos de detección de objetos se dan dos grandes grupos:

- **Modelos de dos etapas:** Aquí encontramos modelos como R-CNN y Faster R-CNN. Lo primero que hacen es calcular las propuestas de Bounding Boxes, después predicen las clases y finalmente refinan las coordenadas de las cajas.
- **Modelos de una sola etapa:** Aquí encontramos YOLO o SSD. Predicen tanto las clases como las Bounding Boxes en un solo paso de la red, teniendo como preferencia la velocidad y eficiencia.

YOLOv8 es un modelo de una sola capa, por lo que es rápido y preciso y está ya optimizado para detectar objetos pequeños, por lo que es ideal para imágenes satelitales.

2.4. Entrenamiento

En el entrenamiento del modelo de detección de objetos es donde la red neuronal, ya creada en nuestro caso, aprende a clasificar y localizar objetos a partir de un dataset proporcionado con imágenes ya anotadas (Bounding Boxes). Varios de los conceptos que son fundamentales de entender en este proyecto para que pueda ser llevado a cabo son:

- **Loss Functions:** La función de pérdida siempre se intenta minimizar. Es una combinación entre el cálculo de la posición de la caja(Localización), la clase del objeto (Clasificación) y la probabilidad de que ese objeto se encuentre en tal posición.
- **Learning Rate:** La tasa de aprendizaje es un hiper parámetro controla el ajuste de los pesos en cada actualización. Si la tasa es demasiado alta el modelo se vuelve inestable, si es demasiado bajo tiene el riesgo de atascarse en el aprendizaje.
- **Epochs:** En cada epoch se recorren todas las imágenes de entrenamiento, ajustando los parámetros de predicción en base a la función de pérdida obtenida de la anterior predicción. Según este nuevo ajuste, se hace una nueva predicción y se obtiene una nueva función de pérdida. Cuanto más epochs “mejor” es el modelo, pero existe el riesgo de ser sobreentrenada y que se especialice más de lo que requiere el objetivo. En este caso, el modelo no dará buenos resultados para entradas generales.
- **Batch size:** El batch es el grupo en el que varias imágenes son procesadas al mismo tiempo. Un dataset de 3000 imágenes no procesa todas individualmente si no que divide este en grupos. Esto permite entrenar con más datos, pues suaviza los datos y hace a el entrenamiento más eficiente computacionalmente. El batch size o tamaño de batch es el número de imágenes que se procesarán en el mismo batch.
- **Shuffle:** Al hacerse shuffle, el orden en el que se procesan las imágenes cambia en cada epoch. La función principal de esto es evitar que el modelo aprenda el patrón que siguen las imágenes del dataset, por lo que previene el sobreajuste y asegura que los batch sean diferentes en cada epoch.
- **Validation:** Tras el entrenamiento, este se evalúa con una serie de métricas.
- **Early y Best Stopping:** Si el modelo deja de mejorar epoch tras epoch se puede detener el aprendizaje. Los modelos que obtienen mejor mAP se suelen guardar aunque el entrenamiento continúe.
- **Data Augment:** Es opcional pero muy usado. Se trata de hacer una serie de transformaciones como rotaciones, escalado, recortes o cambios de brillo y contraste para mejorar la generalización del modelo. Estas transformaciones dependen del caso, la idea tras esto es crear nuevas imágenes que presenten diferencias que se pueden dar naturalmente en los datos

con los que se trabajan. En este caso, como trabajamos con imágenes aéreas, una variación que se puede dar en las imágenes es la de rotación de los elementos que buscamos detectar. Otra es la variación del brillo, ya que dependiendo de la sombra, los elementos y el fondo pueden estar más o menos iluminados.

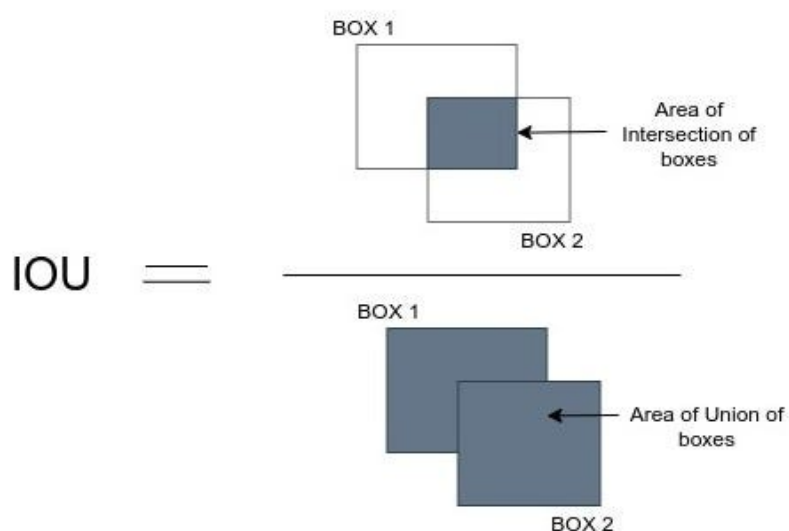
2.5. Métricas y evaluación

Tras la etapa de entrenamiento, es crucial hacer la evaluación de este. Para ellos se suelen usar una serie de métricas. Las más comunes en modelos de detección son:

- **Intersection over Union:** Esta métrica se usa solo en las imágenes de validación y test, nunca en las de entrenamiento. Por cada objeto anotado en la imagen se compara con el Bounding Box de la predicción y se calcula un IoU. Si un objeto generase varios Bounding Boxes en la predicción se tiene en cuenta el mejor IoU. Este se calcula como:

$$IoU = \text{Área de Intersección} / \text{Área de Unión} \quad IoU \in [0, 1]$$

Si $IoU = 1$ los Bounding Boxes coinciden completamente, si $IoU = 0$ entonces no hay superposición. Esta métrica sirve para ayudar a saber si la predicción es correcta, cogiendo un umbral, por ejemplo, $IoU \geq 0.5$. También se usa para calcular métricas como mAP-50.



- **Precision:** Indica qué porcentaje de los objetos detectados son realmente válidos. Por ejemplo, en los que $IoU \geq 0.5$.
- **Recall:** Indica cuántos de los objetos que realmente existen en el dataset han sido detectados. Por ejemplo, en los que $IoU \geq 0.5$.
- **mAP:** Mean Average Precision o promedio de precisión indica el porcentaje de imágenes que superan cierto umbral de IoU. Por ejemplo mAP-50 indica el porcentaje donde $IoU \geq 0.5$, otros como mAP-50-95 hace un cálculo más complejo entre distintas métricas mAP.

- **F1-Score:** Para evitar el engaño por parte de la precisión o el recall se usa una métrica de puntuación para evaluar el desbalance de estas. Si la precisión es alta pero hay un recall bajo, diremos que el modelo es “cobarde”, por el contrario, si el recall es alto pero la precisión es baja diremos que se arriesga demasiado. La fórmula viene definida por:

$$f1 - score = (2 \times Precision \times Recall) / (Precision + Recall)$$

- **Matriz de confusión:** En un modelo de clasificación, es una matriz que representa una comparación entre las predicciones del modelo y los valores reales de los elementos. En este caso, las filas representan las clases predichas y las columnas representan las clases reales. La diagonal representa los aciertos del modelo.

2.7. Aplicación en imágenes satelitales

Como se mencionó brevemente antes, las imágenes satelitales enfrentan una serie de problemas a la hora de implementar un modelo de detección de imágenes sobre ellas:

- Los objetos suelen ser pequeños y están distribuidos muy densamente.
- Encontramos mucha variedad de perspectivas y contextos dónde se encuentran los objetos.
- Se necesitan anotaciones con cierto grado de precisión para hacer un entrenamiento supervisado.

3. IMPLEMENTACIÓN

3.1. Tecnologías

La implementación de este proyecto ha sido llevada a cabo en un notebook, más específicamente en el entorno de Google Colab, donde junto a Python y otras herramientas y librerías se ha trabajado en las distintas partes de esta. Entre las herramientas y librerías usadas están:

- **OpenCV** es una librería de computer vision de código abierto, y que está disponible para muchos lenguajes de programación, entre ellos Python, el cual nosotros utilizaremos.
- **Panda** es una herramienta de análisis, manipulación y manejo de datos estructurados.
- **Matplotlib3** es una librería para crear visualizaciones estáticas, animadas e interactivas en Python. Nosotros usamos Matplotlib versión 3.10 para mostrar las gráficas de las métricas del modelo.

- **Yolov8** es la arquitectura de red neuronal o modelo no entrenado que usamos para entrenar con el dataset y convertirlo en un modelo de detección de objetos.
- **Roboflow** nos permite importar y descomprimir el dataset de su sitio web directamente desde Google Colab.
- **Numpy** es una librería de Python para operar con vectores y matrices.

3.2. Dataset

Para llevar a cabo este proyecto se ha utilizado un dataset [5] de 3704 imágenes. Siendo 3159 de entrenamiento, 293 de validación y 252 de testeo. Es un dataset cuyo objetos ya es anotados, específicamente divididos en 7 clases: 3W, Bus, Coche, Camioneta, Motocicleta, Camión Abierto, Camión. Nosotros las renombramos como 3W, Bus, Coche, Camioneta, Motocicleta, Camión Abierto y Camión respectivamente.

Para poder usar este dataset tuvimos que importarlo y descomprimirlo en el entorno en formato compatible para YOLOv8, en específico para Object Bounding Boxes. Estas imágenes ya vienen anotadas y roboflow nos permite mediante su librería poder hacer todo esto, y en cualquier formato mediante código.

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="#####")
project = rf.workspace("sardar-vallabhkhair-national-institute-if-technology").project("vehicle-detection-wagr7")
version = project.version(3)
dataset = version.download("yolov8-obb")
```

3.3. Entrenamiento

Para hacer el entrenamiento, necesitamos importar un modelo en primer lugar. Con el siguiente código, se implementa el modelo sin entrenar de YOLOv8:

```
model = YOLO("yolov8n.pt") # modelo base ligero
```

En la fase de entrenamiento del modelo enfrentamos varios problemas. Entre ellos están la capacidad y potencia del computador usado para entrenar, o la fiabilidad del mismo dataset con el que entrenamos para que den buenos resultados.

En cuanto al primer problema, lo solucionamos usando recursos de Google Colab, en este caso la GPU-T4, lo que nos limita en tiempo para poder experimentar entrenando de diferentes formas o varias veces el modelo.

En cuanto al segundo, hemos tenido que fijar, algunas veces a base de prueba y error, algunas características para que el dataset proporcionado [9] sea de calidad. Entre estas está el tamaño del

dataset, la calidad de las imágenes y el contexto en la que se encuentran los objetos. El contexto en el que se encuentran los objetos es de vital importancia, pues el entrenamiento únicamente en contextos parecidos puede hacer que el modelo solo se adapte a imágenes tomadas en solo ciertos entornos. Ha pasado en este proyecto, que únicamente por entrenar un dataset que en bastante de las imágenes las anotaciones estaban en vehículos grandes, mientras algunos vehículos como coches se han dejado sin anotar, que el modelo aprendió a identificar solo clases como *Bus* y obviar cualquier otro objeto.

3.3.1. Factores de importancia

Como ya hemos comentado en el planteamiento los datos que usamos, la forma en la que entrenamos el modelo y la forma en la que lo validamos es de vital importancia. Para esto, entran varios parámetros en juego a la hora de usar YOLOv8.

- Shuffle: En la implementación, el cambio de orden de las imágenes entre epochs es automática con el método *train()* de YOLOv8.
- Epochs: Pasado en el método *train()*, indica la cantidad de épocas con las que se va a entrenar la red.
- Batch: Como *epochs* se usa en el método *train()* con el parámetro *batch*. Indica el tamaño de batch.
- Tamaño de las imágenes: Se usa el parámetro *imgsz* en *train()* para indicar el renderizado que tienen que tener todas las imágenes a la hora de entrenar el modelo.
- Data augment: Con el parámetro *augment* en *train()*, indicamos que se haga un aumento de datos en el conjunto de entrenamiento tiene transformaciones definidas por defecto, que se pueden cambiar. Este parámetro lo tenemos activado en todos los modelos
Para algunos modelos también añadiremos los parámetros *hsv_v* y *degrees*:
Con el parámetro *hsv_v* se modifica el rango de variación del brillo.
Con el parámetro *degrees* se modifica el rango en ángulos en el que se pueden rotar las imágenes.

Todos estos factores se han investigado a la hora de implementar el modelo con YOLOv8 en Colab. Muchos de estos han ido siendo investigados y usados para solucionar problemas encontrados sobre la marcha en la implementación, otros han sido usados desde el principio.

3.4. Métricas de testeo

- Confianza: La confianza es crucial para indicarle al modelo ya entrenado a partir de que probabilidad debe de considerar un objeto como detectado. Al evaluar el modelo la indicamos dentro de la función *val* con el parámetro *conf*.
- IoU: El parámetro *iou* es pasado para indicar a partir de qué valor de este se debe de contar solo la caja superpuesta con mayor valor como la única. En la ausencia de este o un valor muy

elevado pueden aparecer cajas superpuestas tanto en validaciones (métricas) como en predicciones.

4. EXPERIMENTACIÓN

En esta sección se explicará la experimentación que hemos ido haciendo simultáneamente con la implementación del proyecto. Esta segunda ha ido siendo modificada y se han sacado varias versiones a la vez que se ha ido experimentando y sacando posibles mejoras. El modelo detecta vehículos de diferentes tipos en imágenes satelitales, por lo que en esta sección se mostrarán los parámetros y código que se han utilizado y modificado, los resultados obtenidos de las diferentes versiones de este modelo, los errores detectados y las mejoras hechas.

4.1. Dataset

Las configuraciones y rutas principales del dataset importado, se encuentran definidas en un archivo data.yaml que viene ya en la carpeta importada con este. Para hacer modificaciones tanto de las rutas como de las clases etiquetadas necesitamos crear un dataset.yaml nuevo.

Como explicamos en el apartado 3.2 de la implementación, tuvimos que cambiar los nombres de las clases del dataset, para ordenarlos y ponerlos en Español. Nuestro nuevo dataset.yaml fue definido de la siguiente manera para los tres conjuntos de imágenes y las siete clases:

```
%%writefile dataset.yaml
path: VEHICLE-DETECTION-3
train: train/images
val: valid/images
test: test/images

names:
  0: 3W
  1: Bus
  2: Coche
  3: Camioneta
  4: Motocicleta
  5: Camión Abierto
  6: Camion
```

4.2. Entrenamiento

Como se menciona en el apartado 3, para el entrenamiento hacemos uso del modelo YOLOv8, y este, lo entrenamos con diferentes parámetros para obtener resultados distintos.

```
model.train(  
    data="dataset.yaml",  
    epochs=15,  
    imgsz=640,  
    batch=20,  
    augment=True  
)
```

En el mismo método de la anterior figura, se han ido modificando la cantidad de epochs, batches, el tamaño de renderizado de la imagen, el uso o no de data augment general o incluso de métodos en específicos de este.

4.3. Métricas

Como se menciona en el apartado 3.4, las métricas las dividimos en métricas del conjunto de validación, y métricas del conjunto de test.

Para las métricas de validación, hemos optado por mostrar la media de las métricas más importantes tanto numéricamente como en un gráfico, aprovechando que se evalúan en cada epoch del entrenamiento para poder ver visualmente como han ido progresando estas. Se muestra un ejemplo del código y el resultado numérico y gráfico de la precisión, recall y medias mAP50 y mAP50-95:

```

#MÉTRICAS VALIDACIÓN
#Mostrar las métricas mirando directamente el fichero donde se guardan

import pandas as pd

# Cambia la ruta por tu experimento
df = pd.read_csv("runs/detect/train/results.csv")
print(df.head())

#Mostrar las métricas
metrics = model.val()

print(metrics)

# =====
# MÉTRICAS ADICIONALES EN TEXTO
# =====

print("\n==== MÉTRICAS PRINCIPALES =====")
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
print("Precision media:", metrics.box.mp)
print("Recall media:", metrics.box.mr)

#Graficar métricas

import matplotlib.pyplot as plt

plt.plot(df['epoch'], df['metrics/precision(B)'], label='Precision')
plt.plot(df['epoch'], df['metrics/recall(B)'], label='Recall')
plt.plot(df['epoch'], df['metrics/mAP50(B)'], label='mAP50')
plt.plot(df['epoch'], df['metrics/mAP50-95(B)'], label='mAP50-95')
plt.xlabel('Epoch')
plt.ylabel('Valor')
plt.legend()
plt.show()

```

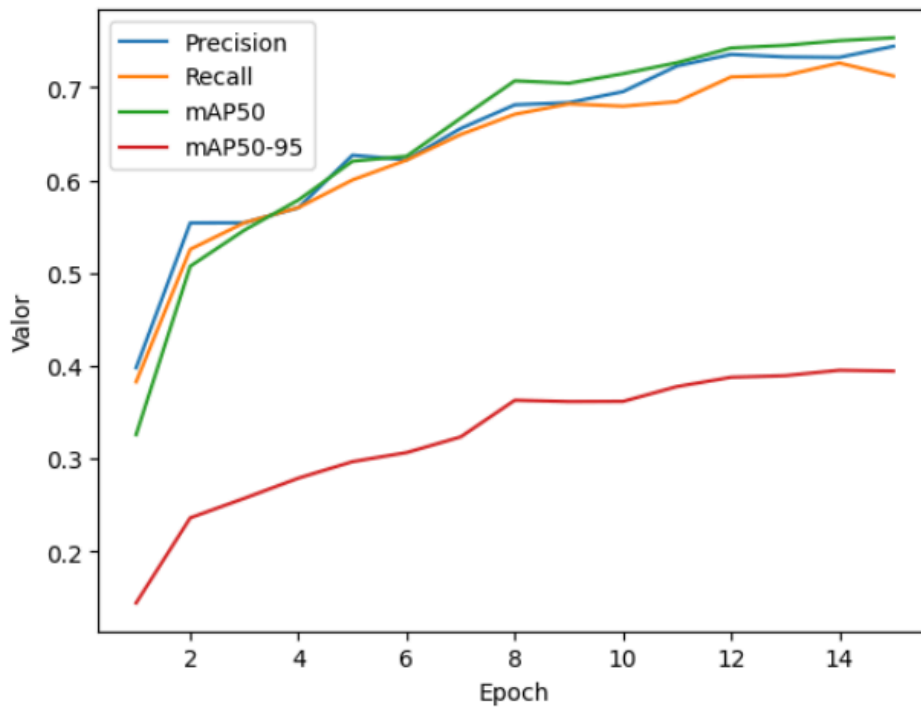
Al usar la función val(), se crea un objeto. Al imprimir las métricas se pueden observar atributos como el índice de clases, el atributo box (que usamos más tarde para graficar la evolución de las métricas) o el resultado de imprimir las métricas, que son varias parejas de arrays, una por cada métrica ('Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)'). El primer array contiene todos los puntos del eje X usados para generar una gráfica automática de esa métrica y el segundo los puntos del eje Y donde se dibujan.

```

ap_class_index: array([0, 1, 2, 3, 4, 5, 6])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7ee43aea8cb0>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
curves_results: [[array([
    0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023, 0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047, 0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071, 0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082, 0.083083, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088, 0.089089, 0.09009, 0.091091, 0.092092, 0.093093, 0.094094, 0.095095, 0.096096, 0.097097, 0.098098, 0.099099, 0.1, 0.10101, 0.102012, 0.103013, 0.104014, 0.105015, 0.106016, 0.107017, 0.108018, 0.109019, 0.11002, 0.111021, 0.112022, 0.113023, 0.114024, 0.115025, 0.116026, 0.117027, 0.118028, 0.119029, 0.12003, 0.121031, 0.122032, 0.123033, 0.124034, 0.125035, 0.126036, 0.127037, 0.128038, 0.129039, 0.13004, 0.131041, 0.132042, 0.133043, 0.134044, 0.135045, 0.136046, 0.137047, 0.138048, 0.139049, 0.14005, 0.141051, 0.142052, 0.143053, 0.144054, 0.145055, 0.146056, 0.147057, 0.148058, 0.149059, 0.15006, 0.151061, 0.152062, 0.153063, 0.154064, 0.155065, 0.156066, 0.157067, 0.158068, 0.159069, 0.16007, 0.161071, 0.162072, 0.163073, 0.164074, 0.165075, 0.166076, 0.167077, 0.168078, 0.169079, 0.17008, 0.171081, 0.172082, 0.173083, 0.174084, 0.175085, 0.176086, 0.177087, 0.178088, 0.179089, 0.18009, 0.181091, 0.182092, 0.183093, 0.184094, 0.185095, 0.186096, 0.187097, 0.188098, 0.189099, 0.1901, 0.191011, 0.192012, 0.193013, 0.194014, 0.195015, 0.196016, 0.197017, 0.198018, 0.199019, 0.2, 0.201011, 0.202012, 0.203013, 0.204014, 0.205015, 0.206016, 0.207017, 0.208018, 0.209019, 0.21002, 0.211021, 0.212022, 0.213023, 0.214024, 0.215025, 0.216026, 0.217027, 0.218028, 0.219029, 0.22003, 0.221031, 0.222032, 0.223033, 0.224034, 0.225035, 0.226036, 0.227037, 0.228038, 0.229039, 0.23004, 0.231041, 0.232042, 0.233043, 0.234044, 0.235045, 0.236046, 0.237047, 0.238048, 0.239049, 0.24005, 0.241051, 0.242052, 0.243053, 0.244054, 0.245055, 0.246056, 0.247057, 0.248058, 0.249059, 0.25006, 0.251061, 0.252062, 0.253063, 0.254064, 0.255065, 0.256066, 0.257067, 0.258068, 0.259069, 0.26007, 0.261071, 0.262072, 0.263073, 0.264074, 0.265075, 0.266076, 0.267077, 0.268078, 0.269079, 0.27008, 0.271081, 0.272082, 0.273083, 0.274084, 0.275085, 0.276086, 0.277087, 0.278088, 0.279089, 0.28009, 0.281091, 0.282092, 0.283093, 0.284094, 0.285095, 0.286096, 0.287097, 0.288098, 0.289099, 0.29, 0.291011, 0.292012, 0.293013, 0.294014, 0.295015, 0.296016, 0.297017, 0.298018, 0.299019, 0.3, 0.301011, 0.302012, 0.303013, 0.304014, 0.305015, 0.306016, 0.307017, 0.308018, 0.309019, 0.31, 0.311011, 0.312012, 0.313013, 0.314014, 0.315015, 0.316016, 0.317017, 0.318018, 0.319019, 0.32, 0.321011, 0.322012, 0.323013, 0.324014, 0.325015, 0.326016, 0.327017, 0.328018, 0.329019, 0.33, 0.331011, 0.332012, 0.333013, 0.334014, 0.335015, 0.336016, 0.337017, 0.338018, 0.339019, 0.34, 0.341011, 0.342012, 0.343013, 0.344014, 0.345015, 0.346016, 0.347017, 0.348018, 0.349019, 0.35, 0.351011, 0.352012, 0.353013, 0.354014, 0.355015, 0.356016, 0.357017, 0.358018, 0.359019, 0.36, 0.361011, 0.362012, 0.363013, 0.364014, 0.365015, 0.366016, 0.367017, 0.368018, 0.369019, 0.37, 0.371011, 0.372012, 0.373013, 0.374014, 0.375015, 0.376016, 0.377017, 0.378018, 0.379019, 0.38, 0.381011, 0.382012, 0.383013, 0.384014, 0.385015, 0.386016, 0.387017, 0.388018, 0.389019, 0.39, 0.391011, 0.392012, 0.393013, 0.394014, 0.395015, 0.396016, 0.397017, 0.398018, 0.399019, 0.4, 0.401011, 0.402012, 0.403013, 0.404014, 0.405015, 0.406016, 0.407017, 0.408018, 0.409019, 0.41, 0.411011, 0.412012, 0.413013, 0.414014, 0.415015, 0.416016, 0.417017, 0.418018, 0.419019, 0.42, 0.421011, 0.422012, 0.423013, 0.424014, 0.425015, 0.426016, 0.427017, 0.428018, 0.429019, 0.43, 0.431011, 0.432012, 0.433013, 0.434014, 0.435015, 0.436016, 0.437017, 0.438018, 0.439019, 0.44, 0.441011, 0.442012, 0.443013, 0.444014, 0.445015, 0.446016, 0.447017, 0.448018, 0.449019, 0.45, 0.451011, 0.452012, 0.453013, 0.454014, 0.455015, 0.456016, 0.457017, 0.458018, 0.459019, 0.46, 0.461011, 0.462012, 0.463013, 0.464014, 0.465015, 0.466016, 0.467017, 0.468018, 0.469019, 0.47, 0.471011, 0.472012, 0.473013, 0.474014, 0.475015, 0.476016, 0.477017, 0.478018, 0.479019, 0.48, 0.481011, 0.482012, 0.483013, 0.484014, 0.485015, 0.486016, 0.487017, 0.488018, 0.489019, 0.49, 0.491011, 0.492012, 0.493013, 0.494014, 0.495015, 0.496016, 0.497017, 0.498018, 0.499019, 0.5, 0.501011, 0.502012, 0.503013, 0.504014, 0.505015, 0.506016, 0.507017, 0.508018, 0.509019, 0.51, 0.511011, 0.512012, 0.513013, 0.514014, 0.515015, 0.516016, 0.517017, 0.518018, 0.519019, 0.52, 0.521011, 0.522012, 0.523013, 0.524014, 0.525015, 0.526016, 0.527017, 0.528018, 0.529019, 0.53, 0.531011, 0.532012, 0.533013, 0.534014, 0.535015, 0.536016, 0.537017, 0.538018, 0.539019, 0.54, 0.541011, 0.542012, 0.543013, 0.544014, 0.545015, 0.546016, 0.547017, 0.548018, 0.549019, 0.55, 0.551011, 0.552012, 0.553013, 0.554014, 0.555015, 0.556016, 0.557017, 0.558018, 0.559019, 0.56, 0.561011, 0.562012, 0.563013, 0.564014, 0.565015, 0.566016, 0.567017, 0.568018, 0.569019, 0.57, 0.571011, 0.572012, 0.573013, 0.574014, 0.575015, 0.576016, 0.577017, 0.578018, 0.579019, 0.58, 0.581011, 0.582012, 0.583013, 0.584014, 0.585015, 0.586016, 0.587017, 0.588018, 0.589019, 0.59, 0.591011, 0.592012, 0.593013, 0.594014, 0.595015, 0.596016, 0.597017, 0.598018, 0.599019, 0.6, 0.601011, 0.602012, 0.603013, 0.604014, 0.605015, 0.606016, 0.607017, 0.608018, 0.609019, 0.61, 0.611011, 0.612012, 0.613013, 0.614014, 0.615015, 0.616016, 0.617017, 0.618018, 0.619019, 0.62, 0.621011, 0.622012, 0.623013, 0.624014, 0.625015, 0.626016, 0.627017, 0.628018, 0.629019, 0.63, 0.631011, 0.632012, 0.633013, 0.634014, 0.635015, 0.636016, 0.637017, 0.638018, 0.639019, 0.64, 0.641011, 0.642012, 0.643013, 0.644014, 0.645015, 0.646016, 0.647017, 0.648018, 0.649019, 0.65, 0.651011, 0.652012, 0.653013, 0.654014, 0.655015, 0.656016, 0.657017, 0.658018, 0.659019, 0.66, 0.661011, 0.662012, 0.663013, 0.664014, 0.665015, 0.666016, 0.667017, 0.668018, 0.669019, 0.67, 0.671011, 0.672012, 0.673013, 0.674014, 0.675015, 0.676016, 0.677017, 0.678018, 0.679019, 0.68, 0.681011, 0.682012, 0.683013, 0.684014, 0.685015, 0.686016, 0.687017, 0.688018, 0.689019, 0.69, 0.691011, 0.692012, 0.693013, 0.694014, 0.695015, 0.696016, 0.697017, 0.698018, 0.699019, 0.7, 0.701011, 0.702012, 0.703013, 0.704014, 0.705015, 0.706016, 0.707017, 0.708018, 0.709019, 0.71, 0.711011, 0.712012, 0.713013, 0.714014, 0.715015, 0.716016, 0.717017, 0.718018, 0.719019, 0.72, 0.721011, 0.722012, 0.723013, 0.724014, 0.725015, 0.726016, 0.727017, 0.728018, 0.729019, 0.73, 0.731011, 0.732012, 0.733013, 0.734014, 0.735015, 0.736016, 0.737017, 0.738018, 0.739019, 0.74, 0.741011, 0.742012, 0.743013, 0.744014, 0.745015, 0.746016, 0.747017, 0.748018, 0.749019, 0.75, 0.751011, 0.752012, 0.753013, 0.754014, 0.755015, 0.756016, 0.757017, 0.758018, 0.759019, 0.76, 0.761011, 0.762012, 0.763013, 0.764014, 0.765015, 0.766016, 0.767017, 0.768018, 0.769019, 0.77, 0.771011, 0.772012, 0.773013, 0.774014, 0.775015, 0.776016, 0.777017, 0.778018, 0.779019, 0.78, 0.781011, 0.782012, 0.783013, 0.784014, 0.785015, 0.786016, 0.787017, 0.788018, 0.789019, 0.79, 0.791011, 0.792012, 0.793013, 0.794014, 0.795015, 0.796016, 0.797017, 0.798018, 0.799019, 0.8, 0.801011, 0.802012, 0.803013, 0.804014, 0.805015, 0.806016, 0.807017, 0.808018, 0.809019, 0.81, 0.811011, 0.812012, 0.813013, 0.814014, 0.815015, 0.816016, 0.817017, 0.818018, 0.819019, 0.82, 0.821011, 0.822012, 0.823013, 0.824014, 0.825015, 0.826016, 0.827017, 0.828018, 0.829019, 0.83, 0.831011, 0.832012, 0.833013, 0.834014, 0.835015, 0.836016, 0.837017, 0.838018, 0.839019, 0.84, 0.841011, 0.842012, 0.843013, 0.844014, 0.845015, 0.846016, 0.847017, 0.848018, 0.849019, 0.85, 0.851011, 0.852012, 0.853013, 0.854014, 0.855015, 0.856016, 0.857017, 0.858018, 0.859019, 0.86, 0.861011, 0.862012, 0.863013, 0.864014, 0.865015, 0.866016, 0.867017, 0.868018, 0.869019, 0.87, 0.871011, 0.872012, 0.873013, 0.874014, 0.875015, 0.876016, 0.877017, 0.878018, 0.879019, 0.88, 0.881011, 0.882012, 0.883013, 0.884014, 0.885015, 0.886016, 0.887017, 0.888018, 0.889019, 0.89, 0.891011, 0.892012, 0.893013, 0.894014, 0.895015, 0.896016, 0.897017, 0.898018, 0.899019, 0.9, 0.901011, 0.902012, 0.903013, 0.904014, 0.905015, 0.906016, 0.907017, 0.908018, 0.909019, 0.91, 0.911011, 0.912012, 0.913013, 0.914014, 0.915015, 0.916016, 0.917017, 0.918018, 0.919019, 0.92, 0.921011, 0.922012, 0.923013, 0.924014, 0.925015, 0.926016, 0.927017, 0.928018, 0.929019, 0.93, 0.931011, 0.932012, 0.933013, 0.934014, 0.935015, 0.936016, 0.937017, 0.938018, 0.939019, 0.94, 0.941011, 0.942012, 0.943013, 0.944014, 0.945015, 0.946016, 0.947017, 0.948018, 0.949019, 0.95, 0.951011, 0.952012, 0.953013, 0.954014, 0.955015, 0.956016, 0.957017, 0.958018, 0.959019, 0.96, 0.961011, 0.962012, 0.963013, 0.964014, 0.965015, 0.966016, 0.967017, 0.968018, 0.969019, 0.97, 0.971011, 0.972012, 0.973013, 0.974014, 0.975015, 0.976016, 0.977017, 0.978018, 0.979019, 0.98, 0.981011, 0.982012, 0.983013, 0.984014, 0.985015, 0.986016, 0.987017, 0.988018, 0.989019, 0.99, 0.991011, 0.992012, 0.993013, 0.994014, 0.995015, 0.996016, 0.997017, 0.998018, 0.999019, 1, 1.001011, 1.002012, 1.003013, 1.004014, 1.005015, 1.006016, 1.007017, 1.008018, 1.009019, 1.01, 1.011011, 1.012012, 1.013013, 1.014014, 1.015015, 1.016016, 1.017017, 1.018018, 1.019019, 1.02, 1.021011, 1.022012, 1.023013, 1.024014, 1.025015, 1.026016, 1.027017, 1.028018, 1.029019, 1.03, 1.031011, 1.032012, 1.033013, 1.034014, 1.035015, 1.036016, 1.037017, 1.038018, 1.039019, 1.04, 1.041011, 1.042012, 1.043013, 1.044014, 1.045015, 1.046016, 1.047017, 1.048018, 1.049019, 1.05, 1.051011, 1.052012, 1.053013, 1.054014, 1.055015, 1.056016, 1.057017, 1.058018, 1.059019, 1.06, 1.061011, 1.062012, 1.063013, 1.064014, 1.065015, 1.066016, 1.067017, 1.068018, 1.069019, 1.07, 1.071011, 1.072012, 1.073013, 1.074014, 1.075015, 1.076016, 1.077017, 1.078018, 1.079019, 1.08, 1.081011, 1.082012, 1.083013, 1.084014, 1.085015, 1.086016, 1.087017, 1.088018, 1.089019, 1.09, 1.091011, 1.092012, 1.093013, 1.094014, 1.095015, 1.096016, 1.097017, 1.098018, 1.099019, 1.1, 1.101011, 1.102012, 1.103013, 1.104014, 1.105015, 1.106016, 1.107017, 1.108018, 1.109019, 1.11, 1.111011, 1.112012, 1.113013, 1.114014, 1.115015, 1.116016, 1.117017, 1.118018, 1.119019, 1.12, 1.121011, 1.122012, 1.123013, 1.124014, 1.125015, 1.126016, 1.127017, 1.128018, 1.129019, 1.13, 1.131011, 1.132012, 1.133013, 1.134014, 1.135015, 1.136016, 1.137017, 1.138018, 1.139019, 1.14, 1.141011, 1.142012, 1.143013, 1.144014, 1.145015, 1.146016, 1.147017, 1.148018, 1.149019, 1.15, 1.151011, 1.152012, 1.153013, 1.154014, 1.155015, 1.156016, 1.157017, 1.158018, 1.159019, 1.16, 1.161011, 1.162012, 1.163013, 1.164014, 1.165015, 1.166016, 1.167017, 1.168018, 1.169019, 1.17, 1.171011, 1.172012, 1.173013, 1.174014, 1.175015, 1.176016, 1.177017, 1.178018, 1.179019, 1.18, 1.181011, 1.182012, 1.183013, 1.184014, 1.185015, 1.186016, 1.187017, 1.188018, 1.189019, 1.19, 1.191011, 1.192012, 1.193013, 1.194014, 1.195015, 1.196016, 1.197017, 1.198018, 1.199019, 1.2, 1.201011, 1.202012, 1.203013, 1.204014, 1.205015, 1.206016, 1.207017, 1.208018, 1.209019, 1.21, 1.211011, 1.212012, 1.213013, 1.214014, 1.215015, 1.216016, 1.217017, 
```

Dónde maps es un array con las valoraciones de cada clase individualmente (basado en la métrica mAP50-95), names muestra todas las clases, y nt_per_class el total de objetos reales por clase en el dataset.

```
===== MÉTRICAS PRINCIPALES =====  
mAP50: 0.7504099406883186  
mAP50-95: 0.3964137312258615  
Precision media: 0.7331597125686582  
Recall media: 0.7267741309399657
```



Para las métricas de los test, hemos de especificarle al método val() la ruta del dataset definida en dataset.yaml que usaremos para estudiar el modelo.

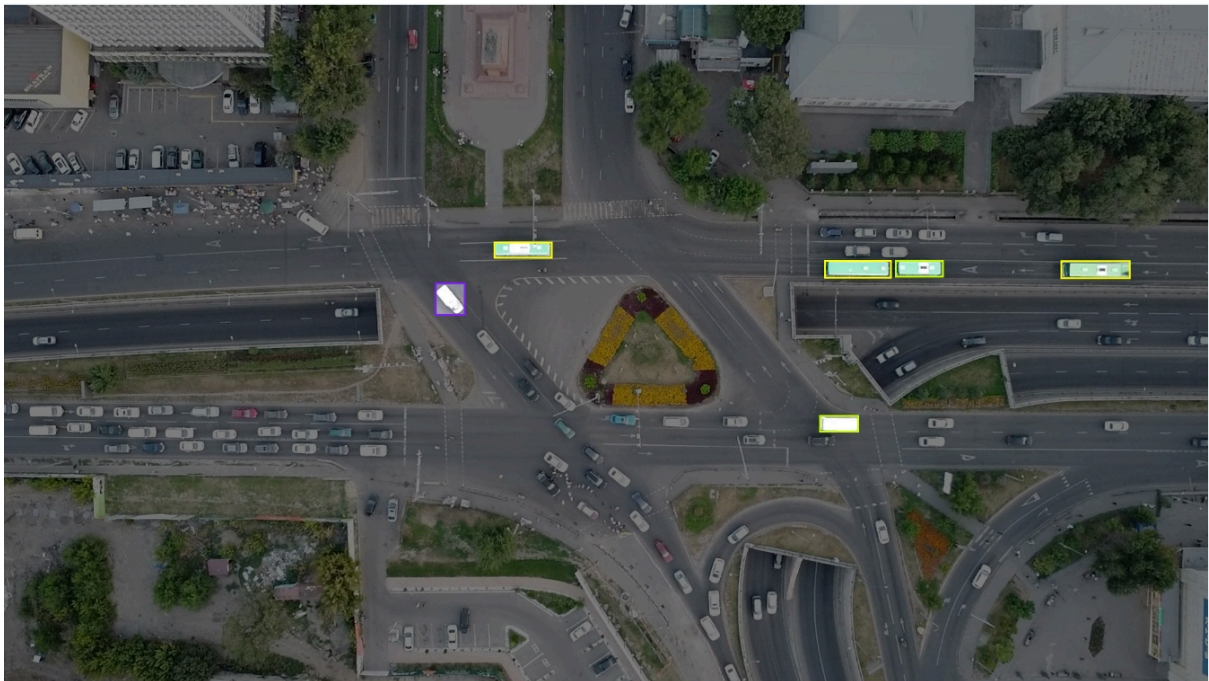
```
#MÉTRICAS TEST  
  
metrics = model.val(data="dataset.yaml", split="test", agnostic_nms=True, iou=0.4, conf=0.001)  
  
print(metrics)  
  
# =====  
# MÉTRICAS ADICIONALES EN TEXTO  
# =====  
  
print("\n===== MÉTRICAS PRINCIPALES =====")  
print("mAP50:", metrics.box.map50)  
print("mAP50-95:", metrics.box.map)  
print("Precision media:", metrics.box.mp)  
print("Recall media:", metrics.box.mr)
```


En este caso se imprimirán de nuevo los resultados de las métricas y el resumen general de estas. No se graficaron debido a que no tenemos resultados diferentes respecto al tiempo como teníamos en el conjunto de validación.

4.4. Versiones de detección de vehículos

Versión Dataset Antiguo:

Usando el dataset [9] con imágenes como la siguiente donde muchas de las imágenes solo contienen etiquetados algunos objetos, mayormente vehículos como camiones, buses o furgonetas, el modelo acaba aprendiendo a no tener en cuenta vehículos de otras clases. Muchas de las imágenes contienen todos los vehículos, pero hay un gran porcentaje de imágenes etiquetadas como la siguiente:



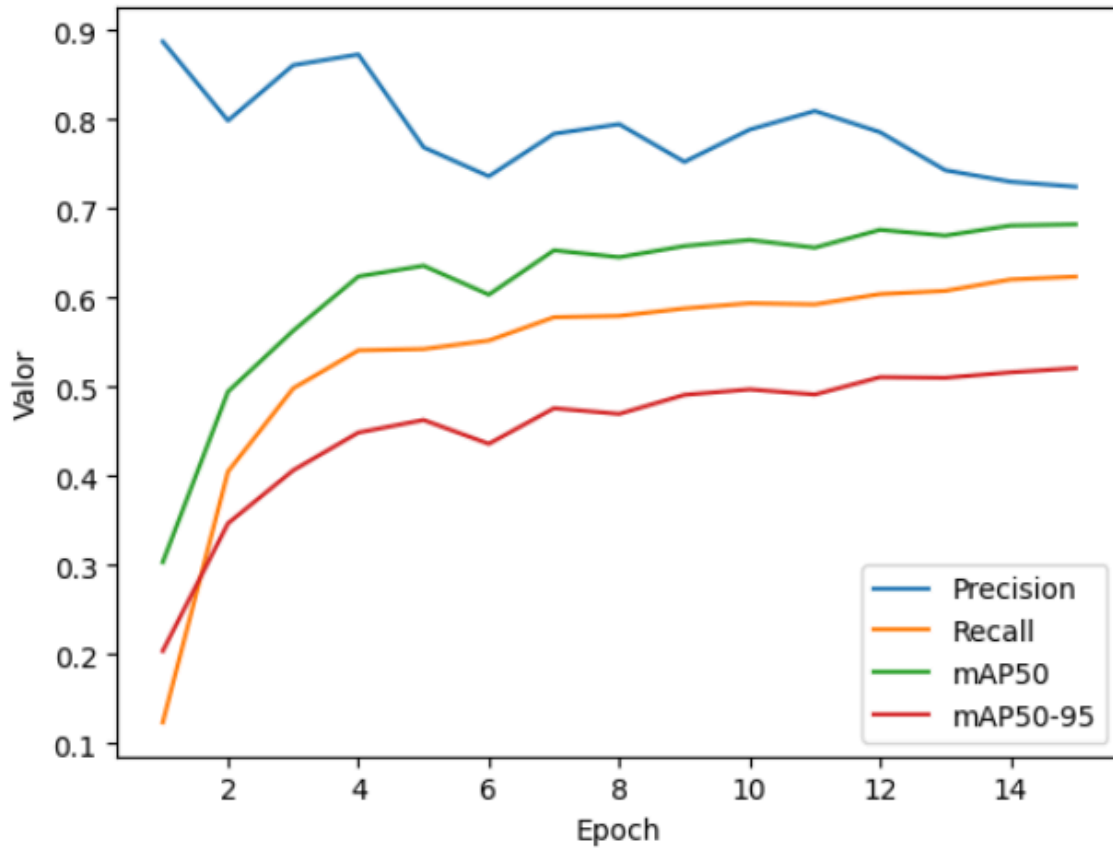
Se muestra como en una imagen cualquiera solo se predice el camión mientras los coches pasan desapercibidos:



La evaluación de las imágenes del test muestran que el recall es bajo, como ya podíamos imaginar por medio de predicciones individuales en imágenes.

Pero algo más curioso ocurre al observar las métricas del conjunto de validación, específicamente la gráfica respecto a los epochs, que nos muestra a diferencia de todas las otras versiones entrenadas como la precisión va siendo peor cuanto más avanza el entrenamiento:

===== MÉTRICAS PRINCIPALES =====
mAP50: 0.6831124592372932
mAP50-95: 0.5233275331911998
Precision media: 0.7278577142092963
Recall media: 0.6239843586655678



Podemos deducir de esto, que cuanto más va aprendiendo el patrón incorrecto de detectar mayormente camiones y no considerar ciertas figuras de las imágenes como coches más errores comete al etiquetar mal y por ende baja la precisión.

Probamos a entrenar el modelo con diferentes epochs y tamaño de batch:

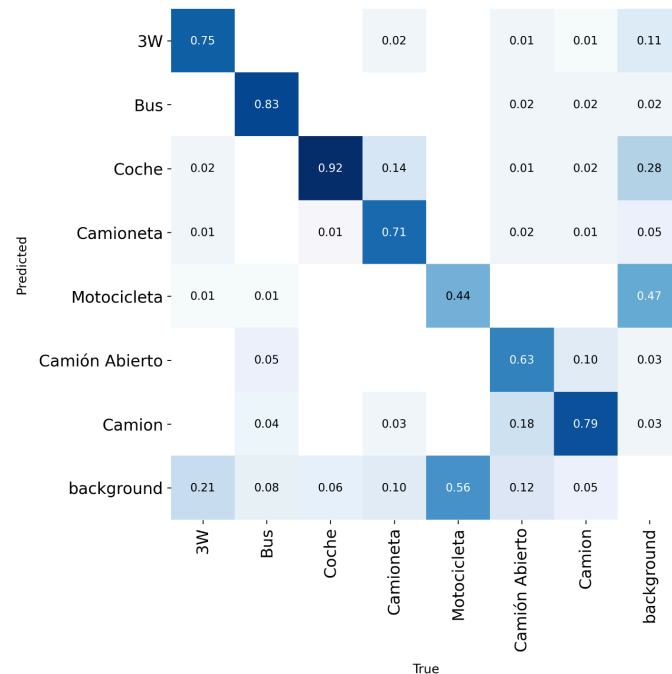
Versión epochs=20, batchsize=10

Precisión: 0.7322309779143846

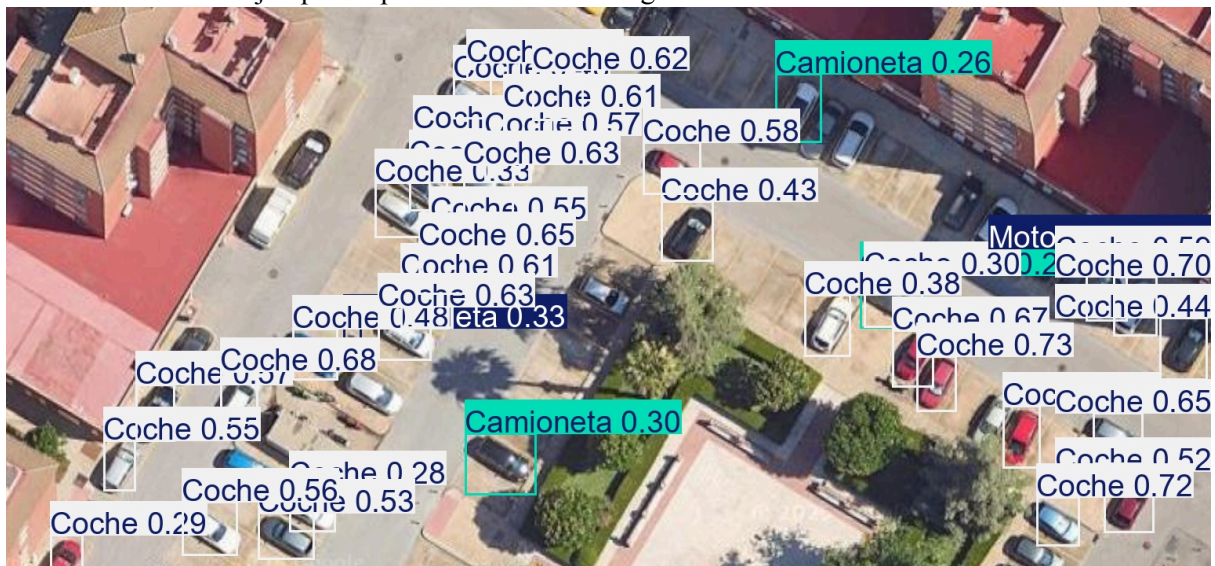
Recall: 0.7302904508316461

F1 score: 0.721103932554634

mAP50-95: 0.40762292680034806



Ejemplo de predicción de una imagen con este entrenamiento:



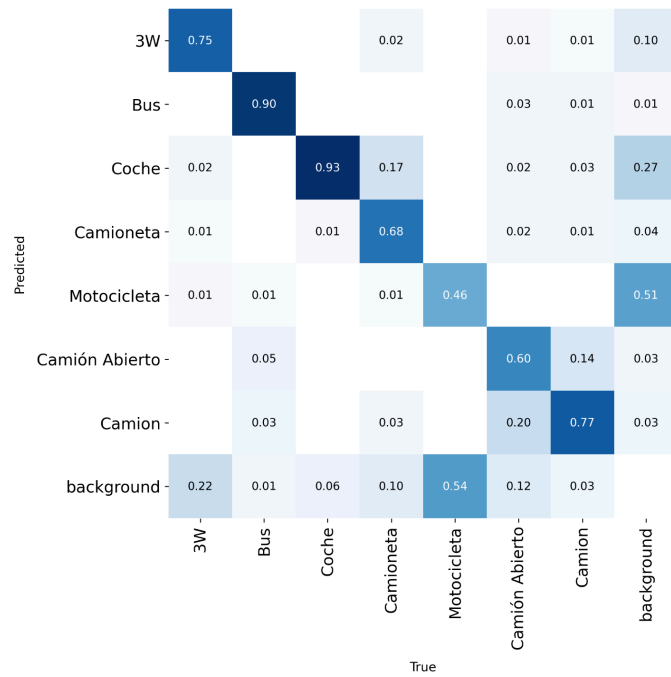
Versión epochs=20, batchsize=25

Precisión: 0.7371159203924007

Recall: 0.750553565273125

F1 score: 0.7343617183823593

mAP50-95: 0.4195186389910343



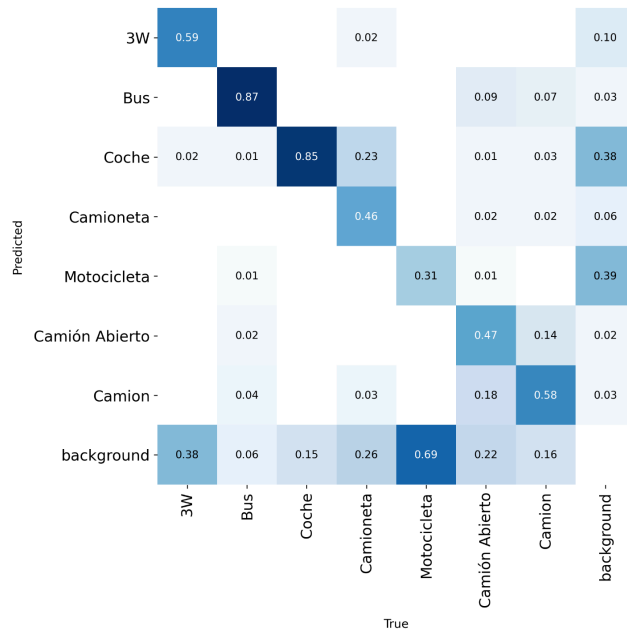
Versión epochs=15, batchsize=25

Precisión: 0.7353996975005437

Recall: 0.7178874863373129

F1 score: 0.7191027793993391

mAP50-95: 0.4006539231891522



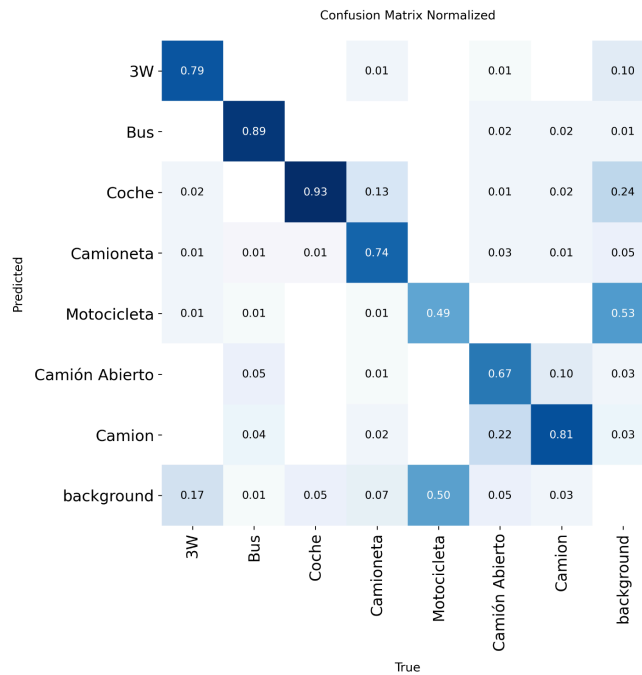
Versión epochs 30, batchsize=8

Precisión: 0.7378976443247127

Recall: 0.7872683028524462

F1 score: 0.7540966315038802

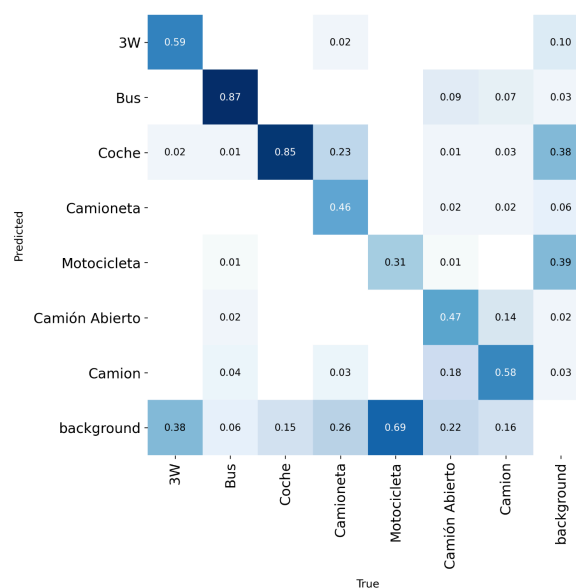
mAP50-95: 0.439908420270166



Para intentar mejorar el rendimiento del modelo, probamos a entrentarlo ajustando las transformaciones utilizadas para la expansión del dataset (Data Augmentation), Utilizando las transformaciones mencionadas en el estudio teórico, datando los como valores a los parámetros mencionados previamente: $hsv_v=0.5$, $degrees=180$.
 Testamos si supone una mejor haciendo dos entrenamientos con estos parámetros y comparando los modelos resultantes con modelos entrenados idénticamente excepto que la expansión del dataset es la que se da por defecto.

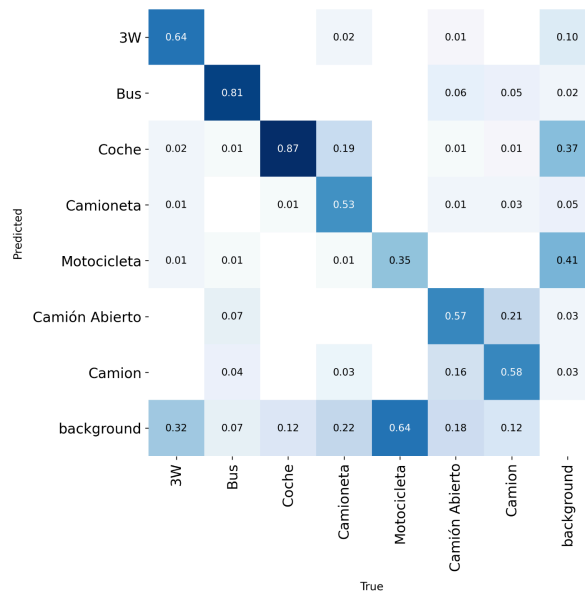
Versión epochs=15, batchsize=25

Precisión: 0.6328081952748177
 Recall: 0.6159481010547773
 F1 score: 0.6093742572804157
 mAP50-95: 0.2925542070589548



Versión epochs=20, batchsize=10

Precisión: 0.6515948247527356
Recall: 0.6184169413322932
F1 score: 0.6194110941344795
mAP50-95: 0.30358888331909817



Podemos comprobar que los resultados son peores a sus contrapartes en las que no se han ajustado estos parámetros. El motivo de esto es que los valores elegidos no han sido los adecuados. El proceso de ajustar estos valores suele ser por prueba y error, en nuestro proyecto hemos descartado esta experimentación, ya que en nuestro caso era más prioritario resolver otros fallos en el rendimiento del modelo.

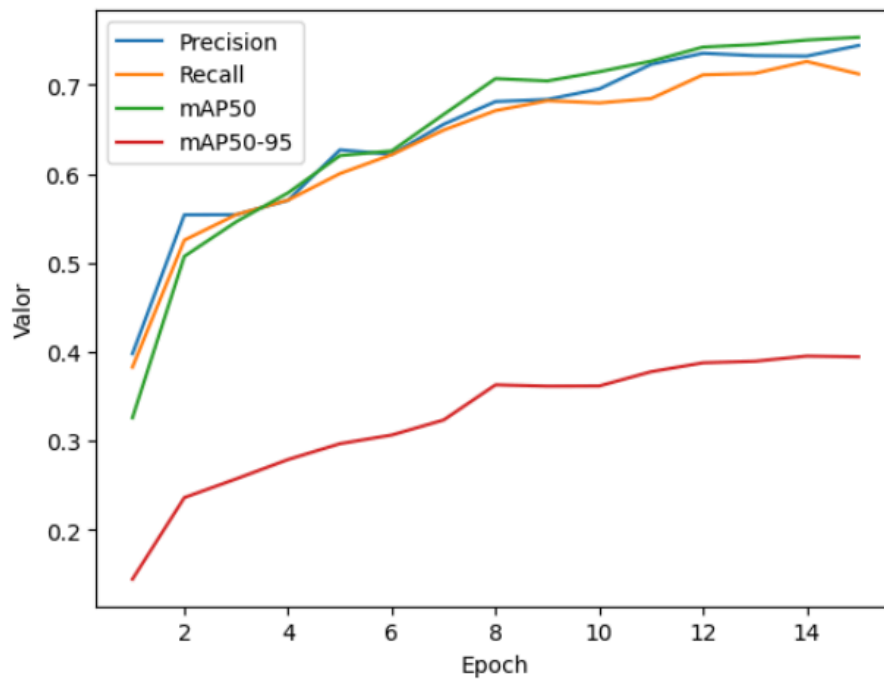
Encontramos, primero, que el rendimiento mejora al aumentar el número de epochs y reducir el tamaño del batch, segundo, el problema de que el modelo detecta múltiples veces un objeto como objetos diferentes, en los siguientes entrenamientos lo atajamos, y por último, que el rendimiento sobre las motos es muy bajo.

Versión X:

Debido a la gran superposición de cajas que estábamos obteniendo en resultados anteriores, decidimos bajar el batch, que es un parámetro que con valores altos puede acrecentar la detección de un objeto varias veces. A esto le sumamos bajar el valor del IoU que necesitaba el modelo para seleccionar únicamente el cajón de mayor valor. Con valores muy altos pueden aparecer varias cajas si ninguna de ellas supera el umbral. Los resultados obtenidos fueron los siguientes:

Métricas de las imágenes de validación (Conf=0.35, IoU=4, epochs=15, batch=30):

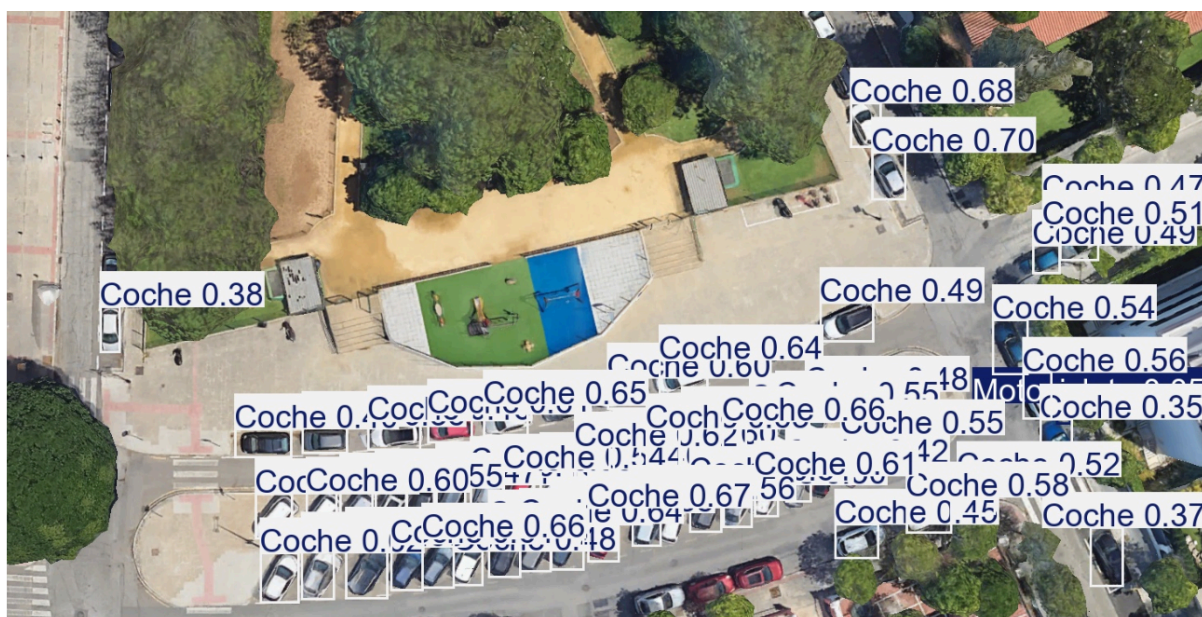
===== MÉTRICAS PRINCIPALES =====
mAP50: 0.7504099406883186
mAP50-95: 0.3964137312258615
Precision media: 0.7331597125686582
Recall media: 0.7267741309399657



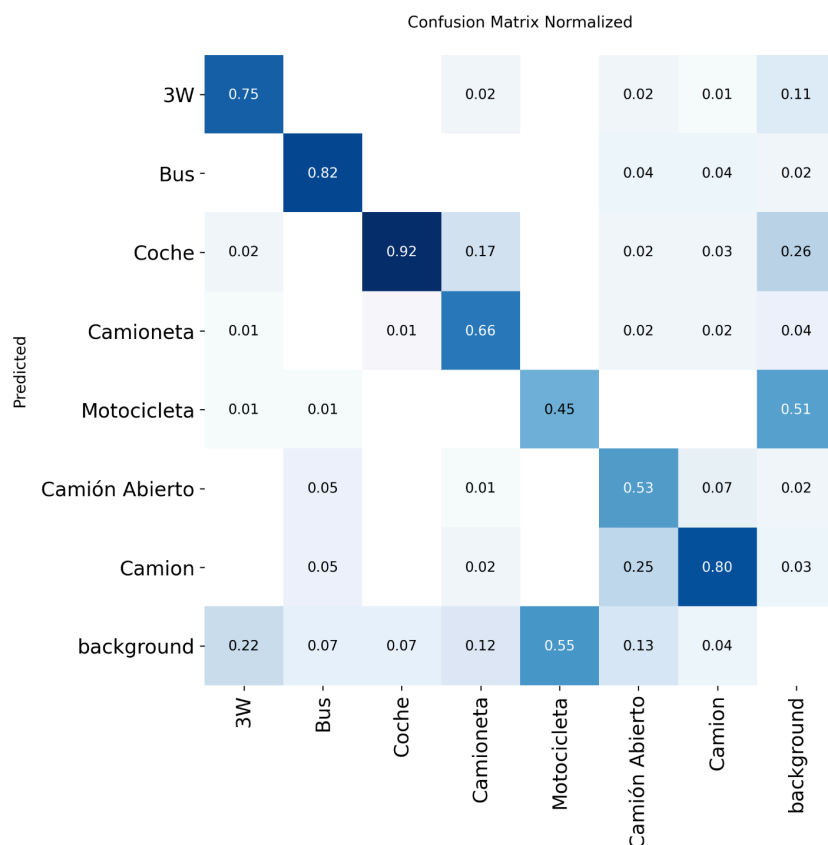
Métricas de las imágenes de test (Conf=0.35, IoU=4, epochs=15, batch=30):

Precisión: 0.7452380737282673
Recall: 0.7062776974140498
F1 score: 0.7165963878349562
mAP50-95: 0.40499871109544217

Predicción en una imagen cualquiera (Conf=0.35, IoU=4, epochs=15, batch=30):



Matriz de confusión (valores normalizados)



Un problema que podemos observar en las matrices de confusión de todos los entrenamientos es el bajo rendimiento en la clase motocicletas. Esto puede ser debido a que, por mala calidad de las imágenes, se confunden las motocicletas con otros elementos del fondo, dando una alta cantidad tanto de falsos positivos como falsos negativos. Para atajar este problema, entrenamos un modelo fijando el tamaño de las imágenes a 1280x1280 píxeles, en vez de el tamaño de 640x640 píxeles que hemos estado usando hasta ahora.

Versión imgsize=1280, epochs=30, batch=8:

Elegimos estos valores de epochs y tamaño del batch tras comparar los entrenamientos que hemos hecho hasta ahora, estos valores son los que mejor resultado nos ha dado hasta el momento. A demás, fijamos IOU a 0.35 y la confianza a 0.45. Los resultados que obtenemos presentan una mejora notable, si comparamos concretamente las métricas sobre la clase motocicleta podemos observar una mejora significativa del rendimiento en comparación con los mejores valores de los entrenamientos anteriores. Sin embargo, hay que mencionar que trabajar con estas dimensiones de imagen es muy costoso, tan solo el entrenamiento ha durado 4 horas, en comparación con los otros entrenamientos que tardan entre 1:30h y 2h, a demás de agotar el GPU disponible. Esta solución es poco eficiente, y no ideal, siendo más deseable ajustar otros aspectos del entrenamiento que no supongan una escalada del coste tan grande.

Precisión: 0.7705572702893517

Recall: 0.8401950249966335

F1 score: 0.8015007966491013

mAP50-95: 0.48144392074510756

4.2. Problemas y mejoras

- Detección únicamente de camiones y vehículos de gran tamaño: En una de las versiones iniciales, el modelo entrenado detectaba vehículos de gran tamaño, dejando la mayoría sin identificar. Este error se debía al uso de un dataset [9] donde a pesar que en muchas imágenes el etiquetado era correcto, en al menos la mitad de este las imágenes solo contenían los camiones o buses etiquetados. Esto hizo que la red aprendiese a no etiquetar en muchas ocasiones los vehículos como coches.

Al analizar los resultados, y el dataset de nuevo, detectamos de donde podría venir el error y optamos por seleccionar un nuevo dataset.

- Superposición de Bounding Boxes: Al evaluar el modelo en muchas predicciones eran muchos los vehículos que tenían asignados varias bounding boxes. Esto debido a que se permite la superposición de cajas si no se le indica lo contrario a partir de cierto umbral a YOLO con el parámetro IoU. El batch muy alto es otro responsable de que aumentaran las cajas superpuestas, pero es el IoU lo que lo regula.
- Detección de motocicletas: Uno de los errores que no hemos podido solucionar ha sido el de la detección de motocicletas. No solo no detecta las motocicletas bien si no que en muchas ocasiones detecta objetos de tamaño pequeño como si fuesen estas.
Podemos ver tras imprimir las métricas del test la valoración de las distintas clases:

```
maps: array([ 0.28901,  0.60992,  0.46364,  0.42427,  0.14273,  0.3275,  0.5171])
```

La valoración de la clase moto se mantiene bajísima en todos los modelos. En la figura del ejemplo es 0.14.

Tras investigar acerca de la poca eficiencia del modelo con las motocicletas hemos llegado a la conclusión de que se puede deber al renderizado de las imágenes. Las motocicletas son objetos que en los dataset ya son chicos, con pocos pixeles. Estos pixeles, al ser renderizados de imágenes gigantes a imágenes de apenas 640x640 pixeles pueden quedar como ruido.

```
maps: array([ 0.28901, 0.60992, 0.46364, 0.42427, 0.14273, 0.3275, 0.5171])
names: {0: '3W', 1: 'Bus', 2: 'Coche', 3: 'Camioneta', 4: 'Motocicleta', 5: 'Camión Abierto', 6: 'Camion'}
nt_per_class: array([ 2972, 364, 8362, 1176, 13488, 311, 632])
nt_per_image: array([251, 125, 251, 182, 252, 95, 145])
```

Vemos así que el modelo puede estar recibiendo al ser entrenado 13488*X píxeles de ruido etiquetados como “Motocicleta” en el dataset.

Para solucionar este problema, probamos a renderizar las imágenes en 1280x1280.

5. CONCLUSIONES

En el desarrollo de este proyecto, se han hecho uso de programas y herramientas que han facilitado mucho este. En primer lugar, ha sido de vital importancia Roboflow, que nos ha permitido encontrar un dataset de más de 3000 imágenes ya anotado y nos ha permitido importarlo y descargarlo de manera rápida y sencilla en formato compatible para nuestro modelo mediante su librería. La otra herramienta fundamental ha sido la librería de YOLOv8, que nos ha permitido usar el modelo base y sus librerías para poder entrenarlo y analizarlo de forma personalizada. Gracias a Google Colab se nos ha facilitado computacionalmente la tarea, principalmente por el uso de su GPU-T4.

El uso de estas tres herramientas o programas junto a otras librerías conocidas como el caso de Numpy, Panda o Matplotlib han evitado que la realización de este proyecto se convirtiera en una tarea de complejidad elevadísima o incluso imposible para equipos con limitaciones de cómputo, infraestructura e información como el nuestro.

En cuanto la tarea principal, la de detectar vehículos mediante un modelo de aprendizaje entrenado, hemos sacado algunas conclusiones:

- Los objetos de tamaño chico, es decir, formado por pocos píxeles, pueden ser tarea difícil si las imágenes con las que se entrena el modelo son más chicas que las originales.
- La detección de objetos es muy dependiente de la etiquetación de estos. Los modelos pueden sobreentrenarse para mal, como en uno de nuestros casos, si se dejan sin etiquetar un porcentaje de veces alguno de los objetos. No vale con etiquetar un subconjunto de clases en algunas imágenes, y otros en otras, a menos que el objetivo fuese la detección por separado. Si un objeto de cierta clase aparece en el conjunto de entrenamiento, este debe de ser etiquetado.
- La detección de objetos es dependiente del entorno con el que se entrene el modelo. Muchos coches pueden no ser detectados por el simple hecho de estar en alguna dirección o posición en particular. Los coches en las esquinas o cercanos a sombras son poco detectados. También son detectados de manera más fácil en nuestro caso los coches sobre asfalto. Los modelos no solo aprenden a detectar ciertos patrones de píxeles (objeto), aprenden a detectarlos junto a otros conjuntos de píxeles (entorno).

Objetivos

1. Obtener datos preprocesados para el entrenamiento y prueba del modelo
2. Modelo de ML entrenado para la detección y clasificación de vehículos
3. Medición de la efectividad del procesamiento de datos elegido
4. Entendimiento del proyecto por parte de los compañeros

6. AUTOEVALUACIÓN

	Julio Nieto Pérez	Pablo	Miguel
Comprensión y dominio (puntuación triple)	Excelente	Excelente	Excelente
Exposición didáctica	Excelente	Excelente	Excelente
Integración del equipo	Excelente	Excelente	Excelente
Implementación			
Objetivos* (puntuación doble)	Excelente	Excelente	Excelente
Aspectos didácticos	Excelente	Excelente	Excelente
Experimentación y conclusiones	Excelente	Excelente	Excelente
Documentación			
Contenidos	Excelente	Excelente	Excelente

Divulgación de los contenidos	Excelente	Excelente	Excelente
Bibliografía / Recursos científicos	Excelente	Excelente	Excelente

7. TABLA DE TIEMPOS

Planificación Inicial

	Julio	Pablo	Miguel
Búsqueda de temática	3h	3h	2h
Investigación y documentación	10h	10h	10h
Prueba de herramientas	8h	8h	9h
Procesamiento y entrenamiento del modelo	12h	12h	12h
Pruebas	10h	10h	10h
Preparación de la presentación	7h	7h	7h

Hojas de tiempo individuales

Miembro	Fecha de la actividad	Duración	Actividad realizada
Julio	5/11	3 h	Búsqueda temática proyecto

	11/11	2 h	Investigación artículos científicos sobre temática del proyecto
	12/11	2 h	Búsqueda datasets para uso en el proyecto
	13/11	1h	Reunión sobre planificación
	14/11	2 h	Seguimiento 1 e investigación implementación yolov8
	18/11	1 h	Procesamiento y buena etiquetación imágenes
	18/11	2 h	Entrenamiento modelo yolov8 en colab
	19/11	2 h	Mejora entrenamiento modelo, cambio de etiquetas y documentación
	19/11	2h	Mejora de entrenamiento modelos e investigación sobre funciones en el entorno de Colab, métricas del entrenamiento y sus métricas y shuffle del dataset.
	20/11	1 h	Reunión
	21/11	2 h	Investigación, y comprobación de métricas usadas en la implementación
	24/11	2 h	Nuevo entrenamiento sin éxito
	26/11	2 h	Eficiencia del entrenamiento, continuación del proyecto y varias dudas.
	28/11	3 h	Documentación
	28/11	2 h	Documentación
	29/11	3 h	Investigación sobre preprocesado, postprocesado e inferencia. Documentación
	3/12	2 h	Entrenamiento del modelo con Epoch=15, batch=40. Mejoras de las métricas del test.
	5/12	5h	Entrenamiento del modelo con Epoch=14, batch=40. Documentación sobre implementación y experimentación. Presentación.
	5/12	2 h	Documentación y seguimiento 4
	8/12	2 h	Entrenamiento y métricas finales con dataset malo
	8/12	6h	Investigación sobre la matrix_confusion, el array maps de valoración, problema con la detección de motocicletas, los arrays de puntos, documentación del proyecto y presentación
	8/12	1,5 h	Reunión y Presentación
Tiempo Total		50,5 h	

Miembro	Fecha de la actividad	Duración	Actividad realizada
Pablo	5/11	3 h	Búsqueda temática proyecto
	11/11	2 h	Investigación artículos científicos sobre temática del proyecto
	12/11	2 h	Búsqueda datasets para uso en el proyecto
	13/11	2 h	Reunión sobre planificación
	14/11	2 h	Seguimiento 1 e investigación implementación yolov8
	17/11	1 h	Investigación de cómo entrenar modelo IA de manera efectiva
	17/11	2 h	Prueba yolov8 en colab
	19/11	2 h	Mejora entrenamiento modelo, cambio de etiquetas y documentación
	20/11	2 h	Reunión
	21/11	1 h	Entrenamiento modelo epoch=10, batch=10
	21/11	2 h	Seguimiento 2 e investigación sobre exportación de modelo
	22/11	2 h	Entrenamiento modelo epoch=20, batch=10
	23/11	3 h	Intento de entrenamiento modelo epoch=30, batch=10 y pruebas con modelos ya exportados
	24/11	2 h	Entrenamiento modelo epoch=20, batch=25 y prueba de dicho modelo
	26/11	2 h	Interpretación métricas y cómo mejorar el entrenamiento
	27/11	3 h	Entrenamiento con yolov11
	28/11	2 h	Entrenamiento modelo a partir de uno ya entrenado
Tiempo Total		35 h	

Miembro	Fecha de la actividad	Duración	Actividad realizada
Miguel	5/11	3h	Búsqueda temática proyecto
	10/11	2h	Investigación artículos científicos sobre temática del proyecto

	13/11	2h	Lectura de la documentación
	13/11	2h	Reunión sobre planificación
	14/11	2 h	Seguimiento 1 e investigación implementación yolov8
	15/11	1,5 h	Arreglar código para importación de datos y entrenamiento modelo
	15/11	0,5h	Creación repositorio github
	20/11	2 h	Reunión
	21/11	2 h	Seguimiento 2
	25/11	1h	Probar integración con github
	27/11	0,5h	Implementar integración con github
	27/11	2h	Búsqueda sobre preprocesado de datos para mejorar el modelo
	28/11	2h	Seguimiento 3
	03/12	2h	Entrenamiento epoch=15, batch=25 con data augmentation con variación de iluminación y rotación de imágenes
	03/12	2h	Entrenamiento epoch=15, batch=25 sin data augmentation
	03/12	1h	Creación de métricas de las predicciones sobre el conjunto de test
	04/12	1h	Llamada compañero para coordinarnos
	05/12	2h	Entrenamiento modelo 20 epochs con tamaño de batch 10 y data augmentation, con métricas
	05/12	2h	Seguimiento 4
	07/12	2h	Entrenamiento modelo
	08/12	3h	Preparación de métricas y obtención de las métricas de todos los modelos que hemos entrenado.
	08/12	1h	Limpieza de un notebook de obtención de métrica, subir modelos y notebook a github
	08/12	3h	Añadir métricas a la documentación , ajustes de las métricas obtenidas.
	08/12	4h	Entrenamiento modelo 30 epochs, con tamaño de batch 8 y img size 1280, y con iou= 0.35 confianza=0.45
Tiempo Total		46,5 h	

References

- [1] “ultralytics/ultralytics: Ultralytics YOLO 🚀.” n.d. GitHub. Accessed November 13, 2025.
<https://github.com/ultralytics/ultralytics>.
- [2] Bin Xu, Yinjuan Gu, and Bin Wang. n.d. “Vehicle Detection in Aerial Images Using Modified YOLO.”
- [3] Wang, Yuhe, Feng Pan, ZhenXu Li, XiuLi Xin, and WeiXing Li. n.d. “CoT-YOLOv8: Improved YOLOv8 for Aerial images Small Target Detection.” *IEEE*. Accessed 2024.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10451989>.
- [4] Kulkarni, Diya, Shrutika Yeole, Soham Karmarkar, Shubh Gupta, and Vaishnavnee Rathod. n.d. “Robust Vehicle Detection in Aerial Imagery: Comparative Study of YOLO Variants.” *IEEE*. Accessed 2025. <https://ieeexplore.ieee.org/document/11076987>.
- [5] sardar vallabhbhai national institute of technology. n.d. “VEHICLE DETECTION Dataset.”
<https://universe.roboflow.com/sardar-vallabhbhai-national-institute-if-technology/vehicle-detection-wagr7>.
- [6] Matplotlib. Available at: <https://matplotlib.org/>.
- [7] Panda. Available at: <https://pandas.pydata.org/>.
- [8] Opencv. Available at: <https://opencv.org/>.

[9] Dataset Aerial Vehicle Detection

<https://universe.roboflow.com/sovitopencvuniversity/aerial-vehicle-detection-0uc42/images/00DcFdz0RgW39UePc0IY>

[10] Numpy. Available at: <https://numpy.org/>